Source Localization via Conical Least Squares

Parsa Dini

September 2026

This is only a note

1 Introduction

Source Localization has become an important research topic with applications in 5th and 6th generation (5G and 6G) wireless networks, integrated sensing and communication (ISAC), space-air-ground integrated networks (SAGIN), medical systems, computer vision, robotics, and satellite communications. In many scenarios, assuming that the source lies on a known plane or at a specific altitude allows the problem to be solved easily using Angle of Arrival (AOA) measurements via standard Least Squares formulation. However, in applications requiring high precision or where prior geometric assumptions are unavailable, these simplified models fall short.

In such cases, estimating the source coordinates is no longer a convex problem, as it requires solving a set of non-convex conical least squares problems. In this work, we address this by computing the distance from an arbitrary point to the points on a cone, subsequently formulating the non-convex problem using suitable relaxations. Furthermore, traditional iterative approaches like Newton's method are often unsuitable here, as they frequently fail to converge or get trapped in local minima.

To date, we have considered the noiseless regime and obtained promising results; future work will focus on extending this formulation to noisy regimes.

2 Problem Formulation

Starting from the spatial phase shift formula $\frac{2\pi d \sin\theta}{\lambda} = \Delta\phi \pmod{2\pi}$, the goal is to extract the spatial angle θ. Because real-world phase measurements repeat every 2π radians, the true physical phase difference includes an unknown integer number of full wave cycles represented by k, where $k \in \mathbb{Z}$. Adding this phase unwrapping term yields:

$$\frac{2\pi d}{\lambda} \sin\theta = \Delta\phi + 2\pi k \tag{1}$$

Assume we have n observations from a source at the i-th sensing position $s_i \in \mathbb{R}^3$, with corresponding AOA angle θ_i, and unit heading vector a_i. The heading vector can be expressed in two forms:

$$a_i = \frac{\psi'(t_i)}{\|\psi'(t_i)\|_2} = \begin{bmatrix} \cos(\phi^i_{az})\cos(\phi^i_{el}) \\ \sin(\phi^i_{az})\cos(\phi^i_{el}) \\ \sin(\phi^i_{el}) \end{bmatrix} \tag{2}$$

where $\psi(t_i)$ parameterizes the trajectory path at time t_i(coordinates with respect to the origin), while ϕ^i_{az} and ϕ^i_{el} represent the azimuth and elevation angles at the same time, respectively.

Either the spacing d (distance between antennas) exceeds $\frac{\lambda}{2}$ or not, we may be able to find a way to remove the k-ambiguity and extract the AOA angle. Thus, we obtain n cones from the set $\mathcal{A} = \{\mathcal{C}_1, \ldots, \mathcal{C}_n\}$, where each cone $\mathcal{C}_i$ is characterized by the tuple (a_i, s_i, θ_i) with AOA $\theta_i \in [0, \frac{\pi}{2}]$, unit heading vector a_i, and sensing location $s_i \in \mathbb{R}^3$.

2.1 Cone's geometry

We first characterize the points lying on an arbitrary cone $\mathcal{C}_i$ parameterized by the tuple(a_i, s_i, θ_i). For any point $u \in \mathbb{R}^3$ lying on the i-th cone, the geometric relationship yields:

$$(u-s_i)^T a_i = \cos(\theta_i)\|u-s_i\|_2 \implies (u-s_i)^T a_i a_i^T (u-s_i) = \cos^2(\theta_i)(u-s_i)^T I_3 (u-s_i)$$

which simplifies to:

$$(u-s_i)^T \left(\cos^2(\theta_i) I_{3\times 3} - a_i a_i^T\right)(u-s_i) = 0 \tag{3}$$

This expression demonstrates that the cone surface is a quadratic surface defined by a symmetric matrix.

2.2 The distance between a given point and a cone

In this section, we calculate the distance from an arbitrary point $u \in \mathbb{R}^3$ to a given cone C_i. If we project $u - s_i$ into the axis of the cone and also project $u - s_i$ to the curve of cone, These points will shape a cyclic quadrilateral and .... finally we can show that the distance will be — i skipped the proof for now

$$h_i(u) = d(u, \mathcal{C}_i) = \inf_{y \in \mathcal{C}_i} \|u - y\|_2 = |r_i \cos(\theta_i) - d_i \sin(\theta_i)|$$

2.3 Transforming the Problem into a Semidefinite Program using Relaxations

The signed distance from a point $u \in \mathbb{R}^3$ to the i-th cone is defined as:

$$h_i(u) = |r_i \cos(\theta_i) - d_i \sin(\theta_i)| \tag{4}$$

and the overall optimization problem becomes:

$$\mathcal{L}(u) = \min_{u \in \mathbb{R}^3} \sum_{i=1}^{n} h_i(u)^2 \tag{5}$$

To handle this non-convex formulation, we introduce the lifted vector:

$$v = \begin{bmatrix} u \\ r_1 \\ \vdots \\ r_n \\ 1 \end{bmatrix} \in \mathbb{R}^{n+4} \tag{6}$$

Each signed distance function $g_i(u)$ admits a linear representation with respect to v:

$$h_i(u) = c_i^T v$$

where the parameter vector c_i is defined as:

$$c_i = \begin{bmatrix} -\sin(\theta_i) a_i \\ \cos(\theta_i) e_i \\ \sin(\theta_i) a_i^T s_i \end{bmatrix}$$

Here, $e_i \in \mathbb{R}^n$ denotes the i-th standard basis vector. Defining the cost matrix $C = \sum_{i=1}^{n} c_i c_i^T$, the objective function can be rewritten as:

$$\mathcal{L}(u) = \sum_{i=1}^{n} h_i(u)^2 = \sum_{i=1}^{n} v^T c_i c_i^T v = v^T \left(\sum_{i=1}^{n} c_i c_i^T \right) v = v^T C v$$

Since $g_i(u)$ is linear in v, the squared distance $g_i(u)^2$ is linear in the rank-one outer product matrix $Z = vv^T \in \mathbb{R}^{(n+4)\times(n+4)}$, partitioned as:

$$Z = \begin{bmatrix} uu^T & ur^T & u \\ ru^T & rr^T & r \\ u^T & r^T & 1 \end{bmatrix}$$

Using the cyclic property of the trace operator, we rewrite the objective as:

$$\mathcal{L}(u) = v^T C v = \operatorname{tr}(v^T C v) = \operatorname{tr}(C v v^T) = \operatorname{tr}(CZ) \tag{7}$$

The non-convex rank constraint $\operatorname{rank}(Z) = \operatorname{rank}(vv^T) = 1$ is then relaxed to the positive semidefinite condition $Z \succeq 0$.

The structural identities implied by $Z = vv^T$ yield:

$$\begin{aligned} u &= Z_{1:3,\, n+4}, & uu^T &= Z_{1:3,\, 1:3} \\ r_i &= Z_{i+3,\, n+4}, & r_i^2 &= Z_{i+3,\, i+3} \\ 1 &= Z_{n+4,\, n+4}, & r_i r_j &= Z_{i+3,\, n+4} Z_{j+3,\, n+4} \end{aligned}$$

Using $r_i^2 = (u - s_i)^T P_i (u - s_i)$, we obtain the linear equality constraint:

$$Z_{i+3,\,i+3} = \mathrm{tr}(P_i Z_{1:3,\,1:3}) + s_i^T P_i s_i - 2 Z_{1:3,\,n+4}^T P_i s_i$$

Combining these constraints, the complete Semidefinite Program (SDP) is formulated as:

$$\begin{aligned}
\min_{Z} \quad & \mathrm{tr}(CZ) \\
\text{s.t.} \quad & Z \succeq 0, \\
& Z_{n+4,\,n+4} = 1, \\
& Z_{i+3,\,i+3} = \mathrm{tr}(P_i Z_{1:3,\,1:3}) + s_i^T P_i s_i - 2 Z_{1:3,\,n+4}^T P_i s_i, \quad i = 1, \ldots, n, \\
& Z_{i+3,\,n+4} \geq 0, \quad i = 1, \ldots, n, \\
& a_i^T Z_{1:3,\,n+4} - a_i^T s_i \geq 0, \quad i = 1, \ldots, n.
\end{aligned}$$

However this relaxation is not concise and upon solving this Linear Program we will observe that upon many inputs, the algorithm will not converge to a global minima.

2.4 Rank-One Promotion via Difference-of-Convex (DC) Penalty Continuation

We consider a minimization problem with a linear objective function and convex constraints. The standard Semidefinite Relaxation (SDR) drops the non-convex constraint $\mathrm{rank}(Z) = 1$, but the resulting optimal matrix Z^* may have a rank greater than one.

To enforce rank-one, we penalize the objective to obtain:

$$\mathcal{F}_\mu(Z) = \mathrm{tr}(CZ) + \mu \left(\mathrm{tr}(Z) - \lambda_{\max}(Z)\right) \tag{9}$$

Although $\mathcal{F}_\mu(Z)$ is an improvement over the hard rank constraint, it remains non-convex due to $-\lambda_{\max}(Z)$. To optimize it, we construct a convex surrogate function $\mathcal{G}_\mu(Z \mid Z^{(k)})$ satisfying the majorization conditions:

$$\mathcal{G}_\mu(Z \mid Z^{(k)}) \geq \mathcal{F}_\mu(Z), \quad \forall Z \in \mathbb{S}_+^n, \quad \text{and} \quad \mathcal{G}_\mu(Z^{(k)} \mid Z^{(k)}) = \mathcal{F}_\mu(Z^{(k)}) \tag{10}$$

These surrogate conditions are directly implied by the spectral bounds:

$$(v^{(k)})^T Z v^{(k)} \leq \lambda_{\max}(Z), \quad \forall Z \in \mathbb{S}_+^n, \quad \text{and} \quad (v^{(k)})^T Z^{(k)} v^{(k)} = \lambda_{\max}(Z^{(k)}) \tag{11}$$

Linearizing around the principal eigenvector $v^{(k)}$ of $Z^{(k)}$ yields the convex surrogate:

$$\mathcal{G}_\mu(Z \mid Z^{(k)}) = \mathrm{tr}(CZ) + \mu \left(\mathrm{tr}(Z) - (v^{(k)})^T Z v^{(k)}\right) \tag{12}$$

The iterative scheme proceeds as follows: first, solve the original SDP to obtain $Z^{(0)}$. Next, perform an eigenvalue decomposition (EVD) on $Z^{(0)}$ to extract its principal eigenvector $v^{(1)}$, and solve the convex SDP subproblem of the surrogate function $\mathcal{G}_\mu(Z \mid Z^{(0)})$ to obtain $Z^{(1)}$. This process repeats iteratively ($Z^{(0)} \to v^{(1)} \to Z^{(1)} \to v^{(2)} \ldots$). In practice for data synthesis, 5 steps are sufficing to reach convergence.

3 Experiments

Note that these parameters here are different than the parameters above , these are the parameters names' that was used in the code!

To evaluate the proposed algorithm, we fix two arbitrary endpoints p_0 and p_3 in $\mathbb{R}^3$ and generate a smooth 3D trajectory between them using a randomized cubic Bézier curve parameterization. Specifically, given two fixed positions p_0 and p_3, the path length vector is defined as $d = p_3 - p_0$ with distance $L = \|d\|_2$, establishing the primary displacement direction $u = \frac{d}{L}$.

To construct realistic intermediate control points p_1 and p_2, two random Gaussian vectors $v_1, v_2 \sim \mathcal{N}(0, I_3)$ are sampled and projected onto the orthogonal complement of u to obtain unit normal vectors n_1 and n_2:

$$n_i = \frac{v_i - (v_i^T u)u}{\|v_i - (v_i^T u)u\|_2}, \quad i \in \{1, 2\} \tag{13}$$

The control points are then constructed by offsetting the longitudinal positions along d by factors α_1, α_2 and perturbing them orthogonally with scaling magnitudes $k_1, k_2 \sim \mathcal{U}(K_1, K_2)$:

$$\begin{aligned} p_1 &= p_0 + \alpha_1 d + k_1 L n_1 \\ p_2 &= p_0 + (1 - \alpha_1) d + k_2 L n_2 \end{aligned}$$

The continuous trajectory $\psi(t)$ for $t \in [0, 1]$ sampled at n discrete sensing positions is governed by the cubic Bézier curve:

$$\psi(t) = (1-t)^3 p_0 + 3(1-t)^2 t p_1 + 3(1-t) t^2 p_2 + t^3 p_3 \tag{14}$$

The corresponding derivative $\psi'(t)$ yields the velocity along the trajectory:

$$\psi'(t) = 3(1-t)^2(p_1 - p_0) + 6(1-t)t(p_2 - p_1) + 3t^2(p_3 - p_2) \tag{15}$$

Finally, the normalized unit heading vector a_i at each discrete sensing position $s_i = \psi(t_i)$ is computed directly from the velocity field as: $a_i = \frac{\psi'(t_i)}{\|\psi'(t_i)\|_2}$.